

AgriDirect — Full System Specification (Cursor-Ready Final Document)

This document is the **complete, final, machine-readable specification** for building the entire AgriDirect platform inside **Cursor.ai**. It contains all requirements, flows, data models, blockchain requirements, ledger structure, IPFS formats, user dashboards, APIs, and system logic.

Cursor can now generate code **autonomously** from this specification.

1. SYSTEM OVERVIEW

AgriDirect is a **blockchain-based agricultural supply-chain transparency platform** enabling: - Tamper-proof crop lifecycle storage (farmer → distributor → retailer → consumer) - QR-based verification of all crop data - Transparent pricing at every stage - Immutable ledger of transport, storage, and quality data - IPFS-based certificate storage - Smart-contract verification of every supply-chain stage - Role-based dashboards (5 roles): **Admin, Farmer, Distributor, Retailer, Consumer** - AI assistance for farmers (crop guidance + image-based analysis)

Blockchain is used to guarantee: - Transparency - Traceability - Immutability - Decentralized verification

2. USER ROLES & REGISTRATION REQUIREMENTS

2.1 Farmer Registration Fields

```
fullName
farmName
email
phone
location
password
metamaskAddress
farmingCertificate (file → IPFS)
landRecords: {
  type: ["712 Extract", "Pattadar Passbook", "Title Deed", "Khasra/Khatuni/Ghata No."],
  file: IPFS
}
incomeTaxReturn (optional IPFS)
leaseDeed (optional IPFS)
```

```
bankPassbook (optional IPFS)
paymentQRCodeImage (for receiving payments)
farmingExperienceYears
```

After registration → Admin verifies → Access granted → Dashboard unlocked.

2.2 Distributor Registration Fields

```
fullName
companyName
gstNumber
location
email
phone
password
metamaskAddress
distributorCertificate (IPFS)
paymentQRCodeImage
```

2.3 Retailer Registration Fields

```
fullName
shopName
location
email
phone
password
metamaskAddress
retailerCertificate (IPFS)
paymentQRCodeImage
```

2.4 Consumer Registration Fields

```
fullName
email
phone
password
```

```
location
preferences (diet filters)
```

2.5 Admin Functions

- Verify Farmer Documents
- Verify Distributor Documents
- Verify Retailer Documents
- Approve / Reject any entity
- Audit crop certificates
- View blockchain logs

3. FARMER WORKFLOW

3.1 Add New Crop Product

Farmer must upload:

```
productName
category: ["Fruits", "Vegetables", "Grains", "Other"]
dietTags: [Jain, Swaminarayan, Vegan, Organic, Gluten-free, etc.]
pricePerUnit
quantityAvailable
location
harvestDate
cropImages: [2 images]
cropVideo
moistureContent
proteinContent
pesticidesUsed (list)
soilPH
labReportFile (optional → IPFS)
cropDescription
```

System generates:

```
cropId
farmerId
```

```
blockchainTransactionHash
IPFSHashForDocuments
```

3.2 Select Distributor

Farmer views list → selects distributor →

```
POST /farmer/select-distributor
{
  cropId,
  distributorId,
  proposedPrice,
  quantity
}
```

Distributor gets real-time notification.

3.3 Logistic Dispatch Details (Farmer → Distributor)

Farmer enters:

```
vehicleType
vehicleNumber
driverName
transportCompanyName
transportCost
dispatchTimestamp
```

This is sent to blockchain.



4. DISTRIBUTOR WORKFLOW

4.1 Receive Farmer Request

Distributor can: - **Accept** → proceed to payment - **Reject** → farmer notified

4.2 Checkout from Farmer

Payment options:

```
COD
UPI QR Payment
On-chain payment (Metamask)
```

After payment → `LedgerEntry` created.

4.3 Logistics Arrival Details (Distributor)

Distributor logs:

```
arrivalTimestamp
qualityCheckNotes
receivedQuantity
isColdStorageUsed
coldStorageTemperature (optional)
beforeProcessingWeight
afterProcessingWeight
processingStatus (processed / raw)
```

4.4 Add Product to Distributor Marketplace

Distributor uploads:

```
finalProductImage
distributorMargin
packagingDate
batchId (generated)
distributorName
productName
```

All details stored → added to blockchain.

5. RETAILER WORKFLOW

5.1 Browse Distributor Marketplace & Buy

Retailer can checkout with: - COD - UPI - Metamask

5.2 Logistic Arrival Details (Distributor → Retailer)

Retailer logs:

```
arrivalTimestamp
transportTemperature
logisticsCompanyName
vehicleNumber
finalQuantityReceived
retailerSellingPrice
```

5.3 Publish to Retailer Marketplace

Shown to consumer.

6. CONSUMER WORKFLOW

Consumer sees products and buys using: - COD - UPI

Consumer scanning QR sees: - Full journey (Farmer → Distributor → Retailer) - All prices - Transport logs - Quality logs - Lab reports - Product lifecycle timeline - Final certificate

7. BLOCKCHAIN DESIGN

Blockchain: **Ethereum-compatible**, stored via Hardhat.

7.1 Smart Contract Functions

```
registerCrop()
assignDistributor()
```

```
logisticsDispatch()  
logisticsArrivalDistributor()  
logisticsArrivalRetailer()  
addDistributorProduct()  
addRetailerPricing()  
finalizeCertificate()
```

7.2 Smart Contract Events

```
CropCreated  
DistributorAssigned  
LogisticsDispatched  
LogisticsArrivedDistributor  
LogisticsArrivedRetailer  
DistributorProductAdded  
RetailerProductAdded  
CertificateFinalized
```

8. IPFS STORAGE FORMATS

8.1 Certificate JSON Structure

```
{  
  cropId,  
  farmer: {  
    name,  
    farmName,  
    certificates: [IPFS hashes]  
  },  
  distributor: { details... },  
  retailer: { details... },  
  lifecycle: [  
    { stage, timestamp, data, txHash }  
  ],  
  images: [IPFS hashes],  
  labReports: [IPFS],  
  priceBreakdown: {  
    farmerPrice,  
    transportCost,  
    distributorMargin,  
    retailerPrice  
  }  
}
```

```
}  
}
```

9. DATABASE SCHEMAS (MongoDB)

9.1 Farmer

```
Farmer {  
  fullName,  
  farmName,  
  documents: {...},  
  metamaskAddress,  
  products: [...]  
}
```

9.2 CropBatch

```
CropBatch {  
  cropId,  
  farmerId,  
  distributorId,  
  retailerId,  
  lifecycle: [...],  
  ipfsCertificate,  
  blockchainTxHash  
}
```

9.3 LedgerEntry

```
LedgerEntry {  
  timestamp,  
  actorRole,  
  actorId,  
  stage,  
  data,  
  txHash  
}
```




10. API ENDPOINTS (Cursor-Readable)

Auth & Registration

```
POST /register/farmer
POST /register/distributor
POST /register/retailer
POST /register/consumer
POST /login
```

Farmer APIs

```
POST /farmer/products
POST /farmer/select-distributor
POST /farmer/logistics/dispatch
GET /farmer/marketplace
```

Distributor APIs

```
GET /distributor/requests
POST /distributor/accept
POST /distributor/payment
POST /distributor/logistics/arrival
POST /distributor/products
```

Retailer APIs

```
GET /retailer/marketplace
POST /retailer/buy
POST /retailer/logistics/arrival
POST /retailer/publish
```

Consumer APIs

```
GET /consumer/marketplace  
GET /product/:qr
```

11. QR CODE DATA FORMAT

```
{  
  cropId,  
  ipfsCertificate,  
  blockchainTxHash  
}
```

QR redirects to:

```
/verify/:cropId
```

12. AI ASSISTANT (for Farmer)

Python Flask microservice:

```
POST /ai/analyze-image (image)  
POST /ai/ask (text prompt)
```

Outputs: - disease detection - nutrient deficiency - fertilizer recommendations - crop guidance

13. PAYMENT FLOW

Supported: - UPI QR - COD - Metamask (on-chain)

14. FINAL CERTIFICATE

After retailer publishes product → system generates: - Full lifecycle certificate - Stored on IPFS - QR updated
- Shown to consumer

15. READY FOR CURSOR IMPLEMENTATION

This specification is: - Structured - Detailed - Machine-readable - Covers every detail you described - Ready
for Cursor agents (Claude/Codegen) to implement automatically

END OF FINAL CURSOR SPECIFICATION